Powerful Command-Line Applications in Go

Chapter 1 - Your First Command-Line Program in Go

- To start a new module, run `go mod init github.com/llcawthorne/programName`
- We normally define main in a package named main in a file named main.go
- `os.Stdin` allows you to access standard input
- A scanner defaults to `bufio.ScanLines` but can be set by Split to `bufio.ScanBytes` or `bufio.ScanWords` to split the input up differently
- The `flag` package let's you define and parse command-line flags

```
// The parameters are flag, default if not set, help message
lines := flag.Bool("l", false, "Count lines")
bytes := flag.Bool("b", false, "Count bytes")
flag.Parse()
```

- `flag` returns pointers, so you need to dereference the vars: `*lines`
- failing to call flag.Parse() leaves you with zeroed vars and can be hard to troubleshoot
- `flag` adds a `-h` and `--help` flag to display help for your utility
- You can build your go executable for another platform: `GOOS=windows go build`
- `flag.Usage()` will display usage information and `flag.Usage` can be customized by assigning another function to it

```
	// If user did not provide input file, show usage
	if *filename == "" {
		flag.Usage()
		os.Exit(1)
	}
```

- `flag.Args()` provides all the non-flag arguments passed

Chapter 2 - Interacting with Your Users

- It is common to create a separate package containing the business logic and the command line interface

- You can write errors to STDERR with `fmt.Fprintln(os.Stderr, err)`
- When closing due to error, use `os.Exit(1)` or another non-zero value
- `json.Marshal(obj)` and `json.Unmarshal(file, object)` write objects or restore objects from json files
- Useful test output `t.Errorf("Expected %q, got %q instead.", got, expected)`
- You can error out of a test with `t.Fatalf("Error: %s", err)`
- go modules are using domain.com/path/to/package, like `github.com/llcawthorne/todo/`
- You can setup and teardown test in a function that receives `m *testing.M` instead of the normal `t *testing.T` and calls `result := m.Run()` where the tests should be run
- You can test a CLI implementation by building it and calling exec.Command on the result

```
build := exec.Command("go", "build", "-o", binName)
if err := build.Run(); err != nil {
    fmt.Fprintf(os.Stderr, "Cannot build tool %s: %s", binName, err)
    os.Exit(1)
}
dir, err := os.Getwd() // check for err and t.Fatal if it's not nil
cmdPath := filepath.Join (dir, binName)  // filepath is imported from
"path/filepath"
cmd := exec.Command(cmdPath, args, moreArgs) // exec is imported from "os/exec"
if err := cmd.Run(); err != nil {
    t.Fatal(err)
}
```

- To check the output use `out, err := cmd.CombinedOutput()` instead of Run
- `switch/case` is easy to use with command line flags
- Anything that implements a `String` method satisfies the `Stringer` interface and can be passed to any formatting function that expects a `String`
- `os.Getenv("TODO_FILENAME")` will get the `TODO_FILENAME` environment variable
- `io.Reader` interface requires a `Read` method

```go
// getTask function decides where to get the description for a new
// task from: arguments or STDIN assuming os.STDIN is the reader passed
func getTask(r io.Reader, args ...string) (string, error) {
    if len(args) > 0 {
        return strings.Join(args, " "), nil
    }

    s := bufio.NewScanner(r)
    s.Scan()        // we're scanning one line of input
    if err := s.Err(); err != nil {
        return "", err
    }

    if len(s.Text()) == 0 {
        return "", fmt.Errorf("Task cannot be blank")
    }

    return s.Text(), nil
}
```

- You can pipe to STDIN in your test with `cmdStdIn, err := cmd.StdinPipe()` then `io.WriteString(cmdStdIn, stdinVal)` and `cmdStdIn.Close()`

Chapter 3 - Working with Files in Go

- You can parse args in `main` and then call `run` to orchestrate your program logic making your program easier to test
- `ioutil.WriteFile(outFname string, data []byte, 0644)` writes out to a file
- You can use `io.Reader` and `io.Writer` interfaces to mock tests
- Using *golden files* you keep known good output to compare test output to
- You can store test data in `./testdata` and it will be ignored by Go tools
- `ioutil.Readfile(inputFile string)` can read a file and returns `input, err` where `input` is returned as an array of bytes
- `os.Remove(resultFile)` can be used to cleanup output files
- Use `defer` with `os.Remove` to ensure cleanup on function completion
- `ioutil.TempFile(dir, pattern)` can be used to create tempfiles with Close() and Name() methods. Ex: `temp, err := ioutil.TempFile("", "mdp*.html")`

- You can pass in `bytes.Buffer` to capture output that would've gone to STDOUT if you define your parameter by the `io.Writer` interface
- Use `Fprintln(out, "output")` when writing to an `io.Writer`
- You get the value out of a buffer with its `String` method
- You can `switch` on `runtime.GOOS` to provide OS specific functionality
- `exec.LookPath(cmdName)` looks up `cmdName` on PATH
- `exec.Command(cmdPath, cmdParams...).Run()` executes a command
- Go `html/template` and `text/template` packages are flexible ways to provide custom output

Chapter 4 - Navigating the File System

- `filepath` provides a cross platform way to manipulate paths
- consider making a custom type struct to hold configuration options

```
type config struct {
    // extension to filter out
    ext string
    // min file size
    size int64
    // list files
    list bool
}
```

- Note: `config` gains more fields as the chapter goes on — the test cases below already assume a `del bool` field ("delete files") that isn't shown in the snapshot above.
- `filepath.Walk` will walk a path and execute a `walkFn` on each file
- `walkFn` has type `func(path string, info os.FileInfo, err error)`
- *table-driven testing* can be done by defining your test cases as a `slice` of anonymous `struct` containing the data required to run your tests and the expected results. You then iterate over the `slice` using loops to execute all test cases without repeating code

```
testCases := []struct {
    name        string
    cfg         config
    extNoDelete string
    nDelete     int
    nNoDelete   int
    expected    string
}{
    {name:          "DeleteExtensionNoMatch",
     cfg:           config{ext: ".log", del: true},
     extNoDelete:   ".gz", nDelete: 0, nNoDelete: 10,
     expected: ""},
    {name:          "DeleteExtensionMatch",
     cfg:           config{ext: ".log", del: true},
     extNoDelete:   "", nDelete: 10, nNoDelete: 0,
     expected: ""},
    {name:          "DeleteExtensionMixed",
     cfg:           config{ext: ".log", del: true},
     extNoDelete:   ".gz", nDelete: 5, nNoDelete: 5,
     expected: ""},
}

for _, tc := range testCases {
    t.Run(tc.name, func(t *testing.T) {
        var buffer bytes.Buffer

        tempDir, cleanup := createTempDir(t, map[string]int{
            tc.cfg.ext:         tc.nDelete,
            tc.extNoDelete:     tc.nNoDelete,
        })
        defer cleanup()

        if err := run(tempDir, &buffer, tc.cfg); err != nil {
            t.Fatal(err)
        }

        res := buffer.String()

        if tc.expected != res {
            t.Errorf("Expected %q, got %q instead\n", tc.expected, res)
        }

        filesLeft, err := ioutil.ReadDir(tempDir)
        if err != nil {
            t.Error(err)
```

```
        }

        if len(filesLeft) != tc.nNoDelete {
            t.Errorf("Expected %d files left, got %d instead\n",
                tc.nNoDelete, len(filesLeft))
        }
    })
}
```

- You can fetch a files attributes with `os.Stat`
- You can define test helpers by calling `t.Helper`
- You can either return a cleanup function or register one with t.Cleanup

```
func createTempDir(t *testing.T,
    files map[string]int) (dirname string, cleanup func()) {

    t.Helper()

    tempDir, err := ioutil.TempDir("", "walktest")
    if err != nil {
        t.Fatal(err)
    }

    for k, n := range files {
        for j := 1; j <= n; j++ {
            fname := fmt.Sprintf("file%d%s", j, k)
            fpath := filepath.Join(tempDir, fname)
            if err := ioutil.WriteFile(fpath, []byte("dummy"), 0644); err != nil
{
                t.Fatal(err)
            }
        }
    }

    return tempDir, func() { os.RemoveAll(tempDir) }
}
```

- `os.RemoveAll` is useful to remove a tempDir and its contents
- `log` contains logging functions that by default write to STDERR
- `log.New` can be used to write to a different destination. Ex:

```
// log.New(out io.Writer, prefix string, flag int) *Logger
delLogger := log.New(os.Stdout, "DELETED FILE: ", log.LstdFlags)
// a logger is accessible through helper functions
// Print[f|ln], Fatal[f|ln], and Panic[f|ln]
delLogger.Println("test output")
```

- standard log output includes the date and time of the log message
- split a buffer on `\n` with `bytes.Split(buffer.Bytes(), []byte("\n"))`
- `compress/gzip` provides functions to compress data
- `gzip.Writer` implements the `io.Writer` interface and can be created with `gzip.NewWriter`
- Use your `gzip.NewWriter` with `io.Copy(zipWriter, inFile)`. Ex:

```
// Set metadata on the zipWriter
zipWriter.Name = filepath.Base(path)
if _, err = io.Copy(zipWriter, inFile); err != nil {
    return err
}
```

- `os.MkdirAll(targetPath)` creates all necessary directories for `targetPath` and does nothing if the directories already exist. Ex:

```
if err := os.MkdirAll(filepath.Dir(targetPath), 0755); err != nil {
    return err
}
```

- `filepath.Glob` can find all file names from a directory matching a regex. Ex:

```
pattern := filepath.Join(tempDir, fmt.Sprintf("*%s", tc.cfg.ext))
expfiles, err := filepath.Glob(pattern)
if err != nil {
    t.Fatal(err)
}
```

Chapter 5 - Improving the Performance of Your CLI Tools

- Error values can be useful as predefined error conditions

- You can return a new error (`fmt.Errorf`) wrapping your predefined errors
- `encoding/csv` contains functions to read CSV's like ReadAll() and Read()

```
cr := csv.NewReader(r)
allData, err := cr.ReadAll()    // returns data as [][]string
```

- You can use `errors.is` to compare a err to a value and it will match a wrapped err
- expErr can be a useful value within your test anonymous []struct
- `time` is the most basic benchmarking tool
- You can define a benchmark as `func BenchmarkFunc(b *testing.B)`
- After setup in a benchmark, call b.ResetTimer()
- `b.N` holds the number of benchmark runs and is good to loop against
- `ioutil.Discard` is an `io.Writer` that discards the output values
- Run a benchmark with `go test -bench . -run ^$`
- Run it 10x with `go test -bench . -benchtime=10x -run ^$`
- To run with the CPU profiler: `go test -bench . -benchtime=10x -run ^$ -cpuprofile cpu00.pprof` then analyze results with `go tool pprof cpu00.prof`
- `top` and `top -cum` are useful profile commands
- `list funcName` in the profiler will drill down to lines of the function
- The `web` profiler command displays a graph and requires graphviz
- To build a memory profile: `go test -bench . -benchtime=10x -run ^$ -memprofile mem00.pprof`
- To see allocated memory about building a memory profile run `go tool pprof -alloc_space mem00.pprof`
- You can benchmark memory with `go test -bench . -benchtime=10x -run ^$ -benchmem`
- Piping your benchmarks through `tee` is a good way to save the output
- You can compare two benchmarks with `benchcmp`, available from `go install golang.org/x/tools/cmd/benchcmp`
- You can create a trace with `go test -bench . -benchtime=10x -run ^$ -trace trace01.out` then view results with `go tool trace trace01.out`
- goroutines and channels make it easy to process concurrently
- A `WaitGroup` provides a way to coordinate goroutines. Declare your `WaitGroup` with `wg := sync.WaitGroup{}`. Add 1 with `wg.Add()` to the `WaitGroup` for each goroutine and subtract 1 with `wg.Done()` when one finishes

- You can defer a call to `wg.Done()` as you start a goroutine
- It is common to use a channel with an empty struct{} to signal completion
- A `select` is like a `switch` on channels
- Use `runtime.NumCPU()` to get the number of CPU's you are running on

Chapter 6 - Controlling Processes

- `exec.Cmd` from `os/exec` provides ways to execute commands

```
args := []string{"build", ".", "errors"}
cmd := exec.Command("go", args...)
cmd.Dir = proj
if err := cmd.Run(); err != nil {
    return fmt.Errorf("'go build' failed: %s", err)
}
```

- Define errors values like `var ErrValidation = errors.New("Validation failed")`
- Or define a custom error type like `stepErr`, anything with an `Error` method

```
type stepErr struct {
    step    string
    msg     string
    cause   error
}
func (s *stepErr) Error() string {
    return fmt.Sprintf("Step: %q: %s: Cause: %v", s.step, s.msg, s.cause)
}
func (s *stepErr) Is(target error) bool {
    t, ok := target.(*stepErr)
    if !ok {
        return false
    }
    return t.step == s.step
}
func (s *stepErr) Unwrap() error {
    return s.cause
}
```

- You can define constructors to instantiate types correctly like `newStep`
- You use `context` and `time` to execute a command with a timeout

```
ctx, cancel := context.WithTimeout(context.Background(), s.timeout)
defer cancel()
cmd := exec.CommandContext(ctx, s.exe, s.args...)
if err := cmd.Run(); err != nil {
    if ctx.Err() == context.DeadlineExceeded {
        return ErrCommandTimeout
    }
}
```

- Use `gitExec, err := exec.LookPath("git")` to make sure `git` is in path
- `filepath.Abs` will give you the absolute path of a directory
- An `exec.Command` has a `.Env` property to set environment variables: `gitCmd.Env = append(os.Environ(), g.env...)`
- You can assign `exec.Command` or `exec.CommandContext` to a `var` to later mock the command in tests
- You use `os/signal` to handle signals
- Signals come in on channels: `sig := make(chan os.Signal, 1)`
- `signal.Notify` relays signals on a channel. To relay `SIGINT` and `SIGTERM` just `signal.Notify(sig, syscall.SIGINT, syscall.SIGTERM)`

Chapter 7 - Using the Cobra CLI Framework

- `go get -u github.com/spf13/cobra/cobra` and `go install`
- ~/.cobra.yaml holds your cobra options like author and license
- `cobra init --pkg-name pragprog.com/rggo/ch7/cobra/pScan` to initialize
- Update `cmd/root.go` with a description of your application
- You can include `Version: "0.1"` along with `Long:` and `Short:`
- You can also set a version template:

```
versionTemplate := `{{printf "%s: %s - version %s\n" .Name .Short .Version}}`
rootCmd.SetVersionTemplate(versionTemplate)
```

- `cobra add command` will add command in cmd/command.go
- Use `Args: cobra.MinimumNArgs(1),` to specify minimum arguments to a command
- Use `Aliases: []string{"d"},` to provide a short alias of d to a command

- You can use viper from `github.com/spf13/viper` for config management
- Viper can bind to flag, load environment variables, or use a config file
- `cobra add completion` will generate a completion command for bash completion
- `cobra add docs` will add a command to generate docs for your app, but you'll need to import `github.com/spf13/cobra/doc`

Chapter 8 - Talking to REST APIs

- You can require a locally installed module as a dependency

```
// First initialize your module
cd $HOME/pragprog.com/rggo/ch8/apis/todoServer
go mod init pragprog.com/rggo/ch8/apis/todoServer
go mod edit -require=pragprog.com/rggo/ch2/todo@v0.0.0
go mod edit -replace=pragprog.com/rggo/ch2/todo=../../../ch2/todo
go list -m all
```

- `net/http` contains an `http.Server` that you can use but it's good to set some options

```
s := &http.Server{
    Addr:           fmt.Sprintf("%s:%d", *host, *port),
    Handler:        newMux(*todoFile),
    ReadTimeout:    10 * time.Second,
    WriteTimeout:   10 * time.Second,
}
if err := s.ListenAndServe(); err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(1)
}
```

- There is a `DefaultServeMux` but it is worthwhile to write your own multiplexor

```
// This syntax has changed with more recent Go and is a lot better now
// There are also whole web mini-frameworks for Go like Gin that you can use
func newMux(todoFile string) http.Handler {
    m := http.NewServeMux()
    m.HandleFunc("/", rootHandler)
    return m
}
```

- A handler is just a `func (http.ResponseWriter, *http.Request)`
- Go web handling is very low level as presented in the book, but the main language has been updated to make it more convenient, and new frameworks like Gin make it even better than that
- Writing tests is very repetitive in this book too, but most modern Go projects use `testify` for assertions to reduce boilerplate
- `httpTest.NewServer` takes your mux and stands up a test server

```
// Always defer the returned cleanup function after opening
// url, cleanup := setupAPI(t)
// defer cleanup()
func setupAPI(t *testing.T) (string, func()) {
    t.Helper()
    ts := httptest.NewServer(newMux(""))
    return ts.URL, func() {
        ts.Close()
    }
}
```

- `net/http` includes `http.Get` for GET requests
- Always defer closing the response body after an http call

```
r, err := http.Get(url)
// handle err
defer r.Body.Close()
if body, err := ioutil.ReadAll(r.Body); err != nil {
    t.Error(err)
}
```

- You can access response attributes like `r.StatusCode` and `r.Header.Get("Content-Type")` (`r.Header` is a map) and `r.Method` , `r.URL` , `r.URL.Path` , and `r.URL.Query()` plus simply `r.Body`
- `encoding/json` contains functions for working with json
- To encode a Go struct as json, you need to json tag the fields

```
type todoResponse struct {
    Results todo.List `json:"results"`
}
```

- Marshal your struct with `json.Marshal(todoResponse)` which returns `([]byte, error)`
- To decode json you need `json.NewDecoder(r.Body),Decode(&item)` which decodes into `item` and only returns `err`
- `http` has all the common and uncommon status codes under `http.StatusWhatever` so you don't need numeric values. Ex: `http.StatusOK == 200`
- `http` has convenience methods `http.Get` and `http.Post` but for more complicated requests you need `req, err := http.NewRequest(http.MethodPatch, url, nil)` then `http.DefaultClient.Do(req)`.
- You can set `log` to discard output in tests with:

```
func TestMain(m *testing.M) {
    log.SetOutput(ioutil.Discard)
    os.Exit(m.Run())
}
```

- You can bind a command-line argument to an environment variable with viper

```
rootCmd.PersistentFlags().String("api-root",
    "http://localhost:8080", "Todo API URL")
replacer := strings.NewReplacer("-", "_")
viper.SetEnvKeyReplacer(replacer)
viper.SetEnvPrefix("TODO")
viper.BindPFlag("api-root", rootCmd.PersistentFlags().Lookup("api-root"))
// Then to fetch elsewhere
apiRoot := viper.GetString("api-root")
```

- The default http client is alright but never times out, create your own:

```
func newClient() *http.Client {
    c := &http.Client{
        Timeout: 10 * time.Second,
    }
    return c
}
```

- You can use `text/tabwriter` to nicely format tabular output

```
w := tabwriter.NewWriter(out, 3, 2, 0, ' ', 0)
fmt.Fprintf(w, "%s\t%d\t%s\t\n", done, k+1, v.Task)
w.Flush()
```

- You want to test your client with mock data and only connect to the real API for integration tests
- The mockServer was a useful pattern and in `mock_test.go` and used in `actions_test.go`
- You can tag integration tests to only run with `go test -tags integration` by adding `// +build integration` at the top of the file before the package declaration
- Add `// +build !integration` to tests you don't want ran along with integration tests

Chapter 9 - Developing Interactive Terminal Tools

- First note, with the version of tcell this app uses, it doesn't display correctly in tmux! Run it in a normal iTerm2 window
- You can do state constants like enum's by declaring a set of const values and assigning the first one equal to `iota`
- You can use `time.Ticker` to execute functions at regular intervals
- To use the repository pattern, declare your interface where you are going to use it and a custom type in another file that implements the interface methods

```go
package repository

import (
    "fmt"
    "sync"
    // business logic
)

type inMemoryRepo struct {
    sync.RWMutex    // slices aren't thread safe
    intervals []pomodoro.Interval
}

func NewInMemoryRepo() *inMemoryRepo {
    return &inMemoryRepo{
        intervals: []pomodoro.Interval{},
    }
}

func (r *inMemoryRepo) Create(i pomodoro.Interval) (int64, error) {
    r.Lock()
    defer r.Unlock()

    i.ID = int64(len(r.intervals)) + 1

    r.intervals = append(r.intervals, i)

    return i.ID, nil
}

func (r *inMemoryRepo) Update(i pomodoro.Interval) error {
    r.Lock()
    defer r.Unlock()
    if i.ID == 0 {
        return fmt.Errorf("%w: %d", pomodoro.ErrInvalidID, i.ID)
    }

    r.intervals[i.ID-1] = i
    return nil
}

func (r *inMemoryRepo) ByID(id int64) (pomodoro.Interval, error) {
    r.RLock()
    defer r.RUnlock()
    i := pomodoro.Interval{}
```

```
    if id == 0 {
        return i, fmt.Errorf("%w: %d", pomodoro.ErrInvalidID, id)
    }

    i = r.intervals[id-1]
    return i, nil
}

// Other special purpose methods for this app
// Breaks - fetches the last n breaks
// Last fetches the last interval in the slice
```

- To test our timer, we can instantiate an instance with short (1-3 * Millisecond) times, but make sure to test you are instantiating the times correctly
- Termdash is a terminal dashboard library with a variety of graphical widgets, dashboard resizing, customizable layout, and handling of mouse and keyboard events
- To make a UI, it is good to start with a struct of widgets which includes the widgets' update functions
- You can use anonymous goroutines within functions to update widgets

```go
func newText(ctx context.Context, updateText <-chan string,
    errorCh chan<- error) (*text.Text, error) {

    txt, err := text.New()
    if err != nil {
        return nil, err
    }

    // Goroutine to update text
    // This anonymous goroutine runs and update the text display anytime
    // it receives a string on `updateText` channel. It exits when the
    // context is cancelled (normally when the application closes)
    go func() {
        for {
            select {
            case t := <-updateText:
                txt.Reset()
                errorCh <- txt.Write(t)
            case <-ctx.Done():
                return
            }
        }
    }()

    return txt, nil
}
```

- It is not uncommon for buttons to have their own struct
- A `termdash` layout is represented using a `container.Container`. You can use multiple containers to split the screen and organize the widgets. You can use `container` to split containers resulting in a binary tree layout or use `grid` to define a grid of rows and columns.
- Out example pomodoro application is arranged using `grid`
- `termdash` runs dashboard applications either though `termdash.Run` or `termdash.NewController`. `Run` is easier but also more resource intensive.

Chapter 10 - Persisting Data in a SQL Database

- First a note, all the example code is using query methods that get block indefinitely with no timeout. Use QueryContext, QueryRowContext, PrepareContext, and ExecContext

```go
package main

import (
	"context"
	"database/sql"
	"log"
	"time"
)

func getUserName(db *sql.DB, id int) (string, error) {
	// Create a context that automatically cancels after 3 seconds
	ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
	defer cancel() // Always clean up the context resources

	var name string
	query := "SELECT name FROM users WHERE id = ?"

	// Pass the context directly into the query execution
	err := db.QueryRowContext(ctx, query, id).Scan(&name)
	if err != nil {
		// If the timeout triggered, this returns
context.DeadlineExceeded
		return "", err
	}
	return name, nil
}
```

- Go's `database/sql` package is great for interacting with databases
- You also need a driver for your particular database which you import as _ since you won't use it directly: `_ "github.com/mattn/go-sqlite3"`
- You can install the go sqlite3 driver with `go get github.com/mattn/go-sqlite3` then `go install github.com/mattn/go-sqlite3`. You need a compiler.
- You can use build tags for different databases like `// +build inmemory` and `// +build !inmemory`
- Go's database drivers seamlessly convert between SQL and Go types
- Open a sqlite database with `db, err := sql.Open("sqlite3", dbfile)`
- Prepare a statement with `insStmt, err := r.db.Prepare("INSERT INTO interval VALUES(NULL, ?,?,?,?,?)")` and `defer insStmt.Close()` then execute it with `res, err := insStmt.Exec(var1, var2, var3, var4, var5)`
- `res.LastInsertId()` gives you the ID from the last insert

- Even with database methods, you need a mutex to lock/unlock
- Query for a single result with `row := r.db.QueryRow("SELECT * FROM interval WHERE id=?", id)` then scan it into a struct with `err := row.Scan(&i.ID, &i.StartTime, &i.PlannedDuration, &i.ActualDuration, &i.Category, &i.State)`
- If the last query had no matches, the err is `sql.ErrNoRows`
- Get multiple rows with `Query`

```
r.RLock()
defer r.RUnlock()

stmt := `SELECT * FROM interval WHERE category LIKE '%Break'
ORDER BY id DESC LIMIT ?`

rows, err := r.db.Query(stmt, n)
if err != nil {
    return nil, err
}
defer rows.Close()

data := []pomodoro.Interval{}
for rows.Next() {
    i := pomodoro.Interval{}
    err = rows.Scan(&i.ID, &i.StartTime, &i.PlannedDuration,
        &i.ActualDuration, &i.Category, &i.State)
    if err != nil {
        return nil, err
    }
    data = append(data, i)
}
err = rows.Err()  // check rows.Err() to make sure it didn't finish with error
if err != nil {
    return nil, err
}

return data, nil
```

- SQL provides datatypes like `sql.NullInt64` that can hold an int64 or NULL

```
var ds sql.NullInt64
err := r.db.QueryRow(stmt, filter, day).Scan(&ds)

var d time.Duration
if ds.Valid {
    d = time.Duration(ds.Int64)
}
```

Chapter 11 - Distributing Your Tool with pomo plus notify

- You can access the platform you are running on through `runtime.GOOS`
- It is possible to make build constraints like `// +build linux`
- When using multiple build constrains those separated by commas are AND'ed together and those separate by spaces are treated as OR, so `// +build containers disable_notifications` turns off notifications if you build with the `containers` or `disable_notifications` tags and `// +build !containers, !disable_notifications` turns the feature on if built without `containers` AND without `disable_notifications`
- `go list -f '{{ .GoFiles }}' ./...` will list all the Go files included in your build. It accepts the `-tags` parameter so you can see what files are included with specific tags
- To get a statically compiled small binary, specify `CGO_ENABLED=0 go build`
- You can build for another platform with `GOOS=windows GOARCH=amd64 go build`
- Run `go tool dist list` to get a list of different platform and architecture combinations and run `go env GOOS` and `go env GOARCH` to see what GOOS and GOARCH are naturally set to
- Here is a useful script to cross compile static binaries for a variety of platforms:

```
#!/bin/bash

OSLIST="linux windows darwin"
ARCHLIST="amd64 arm arm64"

for os in ${OSLIST}; do
    for arch in ${ARCHLIST}; do
        if [[ "$os/$arch" =~ ^(windows/arm64|darwin/arm)$ ]]; then continue; fi

        echo Building binary for $os $arch
        mkdir -p releases/${os}/${arch}
        CGO_ENABLED=0 GOOS=$os GOARCH=$arch go build -tags=tag -o
releases/${os}/${arch}/
    done
done
```

- To cross compile binaries that use external C libraries like sqlite, you need a compiler available for that target such as MINGW for Windows using a command like `CGO_ENABLED=1 CC=x86_64-w64-mingw32-gcc CXX=x86_64-w64-mingw32-g++ GOOS=windows GOARCH=amd64 go build`, but you will still need sqlite installed on the target operating system
- To create an especially stripped down application for containers build with `CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -tags=containers` assuming you have a `containers` tag
- You can also choose to build in a container then run in another container

```
FROM golang:1.15 AS builder
RUN mkdir /distributing
WORKDIR /distributing
COPY notify/ notify/
COPY pomo/ pomo/
WORKDIR /distributing/pomo
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -tags=containers

FROM alpine:latest
RUN mkdir /app && adduser -h /app -D pomo
WORKDIR /app
COPY --chown=pomo --from=builder /distributing/pomo/pomo .
CMD ["/app/pomo"]
```

- With the previous docker file you just run `docker build -t pomo/pomo:latest -f containers/Dockerfile.builder .`

- You can also create an extremely small docker image of just the executable. This file would end with the following after the build:

```
FROM scratch
WORKDIR /
COPY --from=builder /distributing/pomo/pomo .
CMD ["/pomo"]
```

- `go get` works a little different from described in the book. `go get` fetches an application or library, but you still need `go build` or `go install` to build or install binaries